

Design and Implementation of a 5-Stage Pipelined Risc-V Processor with Hazard Detection and Forwarding Unit Using Verilog HDL

Mrs. Velaga Suryakala¹, Kanithi Santoshi bharathi²

1. Assistant Professor , Electronics and Communication Engineering Department
Sanketika Vidya Parishad Engineering College (SVPEC), Visakhapatnam, India.

2.M. Tech Student, Electronics and Communication Department
Sanketika Vidya Parishad Engineering College (SVPEC), Visakhapatnam, India.

Abstract - The RISC-V instruction set architecture has emerged as a modern open-source processor architecture for embedded systems, IoT devices, edge computing platforms, and FPGA-based applications. Due to the increasing demand for high-performance and energy-efficient processors, pipelining has become one of the most widely adopted techniques for improving processor throughput and hardware utilization.

Traditional single-cycle and multi-cycle processors execute instructions sequentially, resulting in lower performance and inefficient hardware resource utilization. Although pipelined processors improve instruction throughput, they introduce challenges such as data hazards, control hazards, and structural hazards that may degrade performance.

This project presents the design and implementation of a 5-stage pipelined RISC-V processor using Verilog HDL. The proposed architecture consists of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages. To improve pipeline efficiency, a Forwarding Unit and Hazard Detection Unit are incorporated into the design. The Forwarding Unit minimizes unnecessary stalls by forwarding results directly between pipeline stages, while the Hazard Detection Unit identifies load-use hazards and inserts stalls whenever required.

The processor is simulated and verified using Icarus Verilog and GTKWave. Experimental results demonstrate successful instruction execution, correct pipeline operation, hazard mitigation, and improved processor throughput. The proposed architecture provides a balanced trade-off between hardware complexity and performance, making it suitable for FPGA-based embedded systems, educational processor design, and future RISC-V research platforms.

1.INTRODUCTION

In recent years, the demand for high-performance processors has increased significantly due to rapid developments in embedded systems, Internet of Things (IoT), artificial intelligence, machine learning, edge computing, cloud computing, and real-time applications. Modern electronic systems require processors that can execute large numbers of instructions efficiently while maintaining low power consumption, reduced hardware complexity, and high computational performance. As a result, processor architecture has become one of the most important research areas in computer engineering and digital system design.

The performance of a processor is primarily determined by its ability to execute instructions quickly and efficiently. Traditional single-cycle processors execute one instruction at a time, requiring all instruction operations such as instruction fetch, decode, execution, memory access, and write-back to be completed within a single clock cycle. Although this approach

simplifies processor design, it leads to inefficient hardware utilization and limits the achievable clock frequency. Multi-cycle processors improve resource utilization by dividing instruction execution into multiple clock cycles; however, they still execute instructions sequentially and therefore suffer from limited throughput.

To overcome these limitations, pipelining has emerged as one of the most effective techniques for improving processor performance. Instruction pipelining divides instruction execution into multiple stages, allowing different instructions to be processed simultaneously in different parts of the processor. Similar to an industrial assembly line, pipelining increases instruction throughput by enabling parallel execution of instruction stages. As a result, the processor can complete more instructions per unit time without significantly increasing hardware resources.

A typical RISC processor pipeline consists of five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). During each clock cycle, multiple instructions occupy different pipeline stages, thereby maximizing hardware utilization and significantly improving system performance. However, pipelining introduces several challenges, including data hazards, control hazards, and structural hazards. These hazards can disrupt the smooth flow of instructions through the pipeline and reduce overall performance if not handled properly.

Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed execution. Control hazards arise from branch and jump instructions, where the next instruction address is uncertain until a branch decision is made. Structural hazards occur when multiple instructions compete for the same hardware resource simultaneously. Effective hazard management techniques are therefore essential for achieving high pipeline efficiency.

Among the various processor architectures available today, the Reduced Instruction Set Computer (RISC) architecture has gained significant popularity due to its simplicity, efficiency, and ease of implementation. RISC processors use a small set of optimized instructions that can be executed quickly, making them highly suitable for pipelined architectures. One of the most recent and widely adopted RISC architectures is RISC-V.

RISC-V is an open-source Instruction Set Architecture (ISA) developed at the University of California, Berkeley. Unlike proprietary processor architectures such as ARM and x86, RISC-V provides complete openness and flexibility, allowing researchers, students, and industries to develop custom processors without licensing restrictions. The modular structure of RISC-V enables designers to implement only the required instruction sets, making it suitable for applications ranging from low-power embedded systems to high-performance computing platforms. Due to these advantages, RISC-V has gained

widespread adoption in academia, research institutions, semiconductor industries, and FPGA-based development platforms.

The availability of open-source tools and extensive documentation has made RISC-V an ideal platform for processor research and educational projects. Implementing a pipelined RISC-V processor provides valuable insight into modern processor architecture concepts such as instruction-level parallelism, hazard handling, forwarding mechanisms, branch management, and pipeline optimization techniques.

This project presents the design and implementation of a 5-stage pipelined RISC-V processor using Verilog HDL. The proposed architecture incorporates essential processor components including Program Counter, Instruction Memory, Register File, Arithmetic Logic Unit (ALU), Control Unit, Data Memory, and Pipeline Registers. To improve execution efficiency and minimize performance degradation caused by hazards, a Forwarding Unit and Hazard Detection Unit are integrated into the processor design. The Forwarding Unit reduces pipeline stalls by directly forwarding data between pipeline stages, while the Hazard Detection Unit identifies data dependencies and inserts stalls only when necessary.

The complete processor architecture is modeled using Verilog HDL and verified through simulation using Icarus Verilog and GTKWave tools. Experimental results demonstrate correct pipeline operation, successful hazard mitigation, and improved instruction throughput compared to non-pipelined processor implementations. The proposed design provides an efficient and scalable platform for FPGA-based processor implementation and serves as a foundation for future enhancements such as branch prediction, cache memory integration, superscalar execution, and advanced RISC-V extensions.

2. LITERATURE REVIEW

The design of high-performance processors has been a major area of research in computer architecture for several decades. As the demand for faster computing systems continues to increase, researchers have proposed various architectural techniques to improve processor performance, reduce execution time, and increase hardware utilization. Among these techniques, instruction pipelining has emerged as one of the most successful methods for improving processor throughput.

Early processor architectures were based on sequential execution models, where a single instruction was executed completely before the next instruction could begin. Although these architectures were simple to design and verify, they suffered from poor hardware utilization and limited performance. As computing requirements increased, researchers introduced pipelined processor architectures to overcome these limitations.

The concept of instruction pipelining was extensively studied by Patterson and Hennessy, who introduced the classical five-stage RISC pipeline architecture consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages. Their work demonstrated that multiple instructions could be executed simultaneously in different pipeline stages, significantly improving processor throughput without increasing the complexity of individual functional units.

One of the most influential contributions in processor architecture is the Reduced Instruction Set Computer (RISC) philosophy. RISC processors simplify instruction execution by using a smaller and highly optimized instruction set. This approach reduces hardware complexity and enables efficient pipelined implementations. Several commercial processors such as MIPS, SPARC, ARM, and PowerPC have successfully adopted RISC principles to achieve high performance.

Among the various RISC architectures, the MIPS processor has been widely used in academic research due to its simple instruction format and straightforward pipeline implementation. Many researchers have implemented five-stage pipelined MIPS processors to study instruction-level parallelism, hazard handling techniques, forwarding mechanisms, and branch prediction methods. These studies demonstrated that pipelining can significantly improve processor performance compared to single-cycle and multi-cycle architectures.

In recent years, RISC-V has emerged as a revolutionary open-source instruction set architecture. Developed at the University of California, Berkeley, RISC-V provides an extensible and modular ISA that allows researchers and industries to design custom processors without licensing restrictions. The open-source nature of RISC-V has accelerated research in processor architecture, embedded systems, FPGA implementations, and hardware security.

Several researchers have proposed FPGA-based implementations of RISC-V processors. FPGA platforms provide flexibility, rapid prototyping capabilities, and lower development costs compared to ASIC implementations. FPGA-based RISC-V processors have been successfully deployed in embedded systems, IoT devices, edge computing platforms, and educational environments. These implementations have demonstrated the suitability of RISC-V for both academic and industrial applications.

A major challenge in pipelined processor design is the occurrence of hazards. Hazards reduce pipeline efficiency by introducing stalls and delays. Data hazards occur when an instruction requires data that is still being processed by a previous instruction. Control hazards occur due to branch instructions, while structural hazards arise when multiple instructions compete for the same hardware resource.

To address data hazards, researchers have proposed various forwarding and bypassing techniques. Forwarding allows data produced in later pipeline stages to be directly supplied to earlier stages without waiting for register write-back. Several studies have shown that forwarding significantly reduces pipeline stalls and improves overall processor throughput. Modern pipelined processors widely employ forwarding units to achieve higher performance.

Hazard detection units have also been extensively investigated. These units monitor instruction dependencies and generate stall signals whenever forwarding alone cannot resolve a hazard. Load-use hazards are among the most common situations requiring hazard detection. Research has demonstrated that integrating hazard detection mechanisms improves processor reliability while maintaining efficient pipeline operation.

Branch instructions introduce another major challenge in pipelined processors. Since the outcome of a branch instruction may not be known immediately, the processor may fetch incorrect instructions, leading to performance penalties. To mitigate this problem, researchers have proposed branch prediction techniques. Static branch prediction uses fixed rules to predict branch outcomes, while dynamic branch prediction

employs hardware mechanisms that learn from previous execution history. Modern processors achieve substantial performance improvements through advanced branch prediction algorithms.

Several recent studies have focused on combining pipelining with FPGA-based processor implementations. Researchers have developed configurable RISC-V cores supporting forwarding, hazard detection, branch prediction, and cache memory integration. These architectures achieve higher clock frequencies and improved throughput while maintaining acceptable hardware resource utilization. Experimental results indicate that properly designed pipelined processors can achieve significant performance gains compared to non-pipelined implementations.

Although many existing RISC-V processor implementations provide efficient pipelined execution, several limitations remain. Some architectures lack comprehensive hazard handling mechanisms, resulting in increased pipeline stalls. Others focus primarily on performance improvement while neglecting hardware resource optimization. Additionally, advanced features such as branch prediction and cache integration often increase design complexity and FPGA resource consumption.

The literature survey reveals that an efficient pipelined processor should provide a balanced trade-off between hardware complexity and performance. Integrating forwarding and hazard detection units can significantly improve pipeline efficiency while maintaining moderate hardware utilization. Furthermore, FPGA-based implementation offers an effective platform for validating processor architectures and evaluating their performance characteristics.

Based on the findings from previous research, the proposed work focuses on the design and implementation of a 5-stage pipelined RISC-V processor using Verilog HDL. The architecture incorporates pipeline registers, forwarding logic, and hazard detection mechanisms to improve throughput and reduce execution delays. The processor is verified through simulation using Icarus Verilog and GTKWave, providing a practical demonstration of modern pipelined processor concepts.

Author	Year	Contribution	Limitation
Patterson & Hennessy	2017	Classical 5-stage pipeline architecture	Basic hazard handling
RISC-V Foundation	2019	Open-source ISA architecture	No implementation details
FPGA RISC-V Researchers	2020	FPGA-based processor implementation	Limited forwarding support
Modern RISC-V Studies	2022	Hazard detection and forwarding	Increased hardware complexity
Proposed Work	2026	5-stage pipeline + forwarding + hazard detection	Future work includes branch prediction and cache

3.Body of Paper

3.1 Program counter(PC)

The Program Counter (PC) is one of the fundamental components of the processor. It is responsible for maintaining the address of the current instruction being executed. During each clock cycle, the Program Counter generates the address required to fetch the next instruction from the instruction memory.

In the proposed processor architecture, the Program Counter is implemented as a 32-bit register. During normal operation, the Program Counter increments by four after each clock cycle because RISC-V instructions are 32 bits (4 bytes) wide. The Program Counter can also be updated with branch target addresses when branch instructions are executed.

The Program Counter plays a critical role in controlling the instruction flow through the processor pipeline. Correct operation of the Program Counter ensures sequential instruction execution and proper instruction fetching.

3.2 Instruction Fetch Stage (IF)

The Instruction Fetch stage is the first stage of the pipeline. In this stage, the processor retrieves the instruction from the instruction memory using the address generated by the Program Counter.

The instruction memory stores the program instructions in machine code format. Whenever the Program Counter provides an address, the corresponding instruction is fetched and transferred to the IF/ID pipeline register.

The main objective of the Instruction Fetch stage is to ensure continuous delivery of instructions to the processor pipeline. Efficient instruction fetching contributes significantly to overall processor performance.

Functions of Instruction Fetch Stage:

- Fetch instruction from instruction memory
- Generate next instruction address
- Transfer instruction to IF/ID register
- Support continuous pipeline execution

3.3 IF/ID Pipeline Register

Pipeline registers are essential components in pipelined processor architectures. The IF/ID pipeline register acts as an intermediate storage element between the Instruction Fetch and Instruction Decode stages.

The IF/ID register stores:

- Current instruction

- Program Counter value

These values remain stable during the next clock cycle while the subsequent instruction is being fetched. This mechanism enables multiple instructions to be processed simultaneously in different stages of the pipeline.

The use of pipeline registers improves throughput and supports instruction-level parallelism.

3.4 Instruction Decode Stage (ID)

The Instruction Decode stage interprets the fetched instruction and generates the necessary control signals required for execution.

During this stage:

- Opcode field is decoded
- Source registers are identified
- Destination register is determined
- Control signals are generated

The Control Unit analyzes the instruction format and determines the operation to be performed by the processor.

This stage is responsible for preparing all information required by the Execute stage.

3.5 Register File

The Register File is a collection of 32 general-purpose registers used to store temporary data during program execution.

The RISC-V architecture defines:

x0 – x31

where:

- x0 is permanently zero
- x1–x31 are general-purpose registers

The Register File provides two read ports and one write port.

Functions:

- Read source operands
- Store computation results
- Provide data to ALU

- Receive data during Write Back stage

The Register File significantly reduces memory access operations and improves processor performance.

3.6 Control Unit

The Control Unit acts as the decision-making block of the processor.

It generates control signals based on the instruction opcode.

Control signals generated include:

- RegWrite
- MemRead
- MemWrite
- ALUSrc
- Branch
- ALUControl

The Control Unit coordinates all processor operations and ensures correct execution of instructions.

Without the Control Unit, the processor would be unable to interpret instruction functionality.

3.7 ID/EX Pipeline Register

The ID/EX pipeline register stores all decoded information before it enters the Execute stage.

Stored information includes:

- Read Data 1
- Read Data 2
- Immediate values
- Destination register address
- Control signals

This register separates the Decode stage from the Execute stage and supports parallel instruction processing.

3.8 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit is the computational core of the processor.

The ALU performs arithmetic and logical operations including:

- Addition
- Subtraction
- AND
- OR
- XOR
- Comparison operations

Based on the ALU control signal generated by the Control Unit, the ALU executes the required operation and produces the result.

The ALU output is forwarded to subsequent pipeline stages for further processing.

3.9 Forwarding Unit

Data hazards are one of the major challenges in pipelined processor architectures.

A data hazard occurs when an instruction depends on the result of a previous instruction that has not yet completed execution.

The Forwarding Unit resolves such hazards by directly forwarding data from later pipeline stages to the Execute stage without waiting for the Write Back stage.

For example:

ADD x1, x2, x3

ADD x4, x1, x5

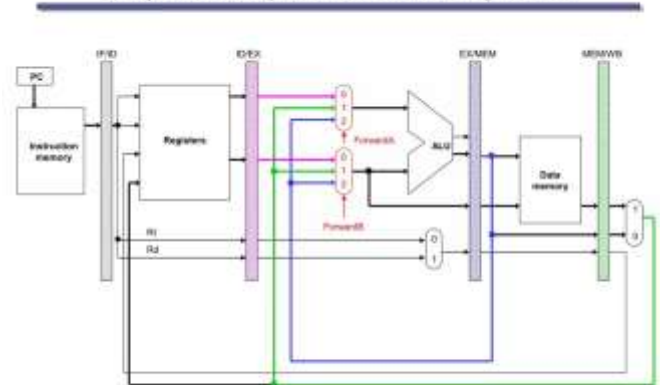
The second instruction requires the result of the first instruction. Instead of stalling the pipeline, the Forwarding Unit directly forwards the result.

Advantages:

- Reduces pipeline stalls
- Improves throughput
- Enhances processor efficiency

The Forwarding Unit is one of the key improvements implemented in the proposed architecture.

Simplified datapath with forwarding muxes



3.10 Hazard Detection Unit

Certain hazards cannot be solved using forwarding alone.

For example:

LW x1, 0(x2)

ADD x3, x1, x4

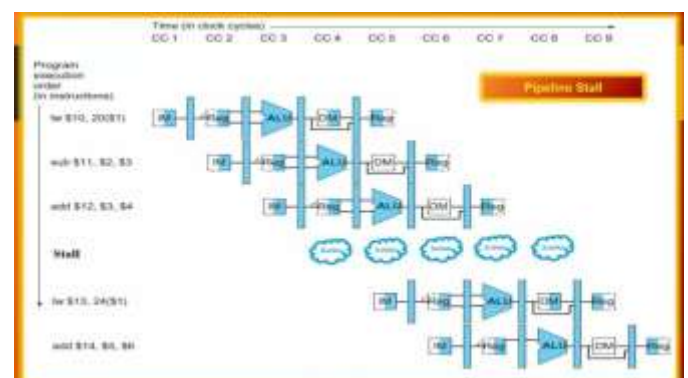
In this case, the required data becomes available only after memory access.

The Hazard Detection Unit monitors instruction dependencies and generates stall signals whenever necessary.

Functions:

- Detect load-use hazards
- Generate stall signals
- Prevent incorrect execution
- Maintain pipeline correctness

This unit ensures reliable processor operation even in the presence of instruction dependencies.



3.11 Execute Stage (EX)

The Execute stage performs actual instruction execution.

Activities include:

- ALU operations
- Address calculations
- Branch condition evaluation
- Forwarded data processing

The Execute stage receives operands from the ID/EX register and produces results that are transferred to the EX/MEM register.

3.12 EX/MEM Pipeline Register

The EX/MEM register stores execution results before they are transferred to the Memory stage.

Stored information:

- ALU result
- Destination register
- Control signals

This register enables concurrent execution of multiple instructions within the pipeline.

3.13 Data Memory Stage (MEM)

The Memory stage handles load and store instructions.

Operations include:

- Reading data from memory
- Writing data into memory
- Memory address validation

For arithmetic instructions, this stage simply forwards ALU results.

For memory instructions, the Data Memory block performs actual memory transactions.

3.14 MEM/WB Pipeline Register

The MEM/WB register stores the final instruction results before they are written back to the Register File.

Stored data includes:

- Memory output
- ALU result
- Destination register

The MEM/WB register acts as the final pipeline buffer.

3.15 Write Back Stage (WB)

The Write Back stage is the final stage of instruction execution.

In this stage:

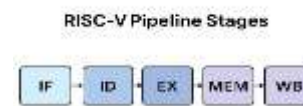
- ALU results are written back to registers
- Memory data is stored in destination registers
- Program execution is completed

The Register File is updated with the final computed result.

This stage ensures that future instructions can access the newly generated data.

3.16 Pipeline Operation

The proposed processor uses a five-stage pipeline architecture:



Multiple instructions are processed simultaneously in different pipeline stages.

Benefits include:

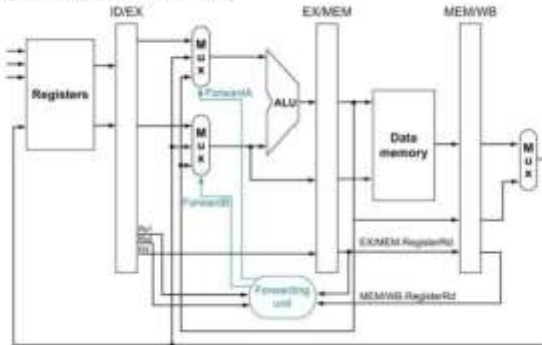
- Increased throughput
- Better hardware utilization
- Reduced execution time
- Improved processor performance

The integration of Forwarding and Hazard Detection Units further improves efficiency by minimizing pipeline stalls and ensuring correct instruction execution.

The proposed architecture successfully demonstrates the implementation of a complete 5-stage pipelined RISC-V processor with enhanced hazard handling capabilities suitable

for FPGA-based embedded systems and educational processor research.

Figure 2: 5-stage pipeline with forwarding



Basic five-stage pipeline

Instr. No.	1	2	3	4	5	6	7
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX

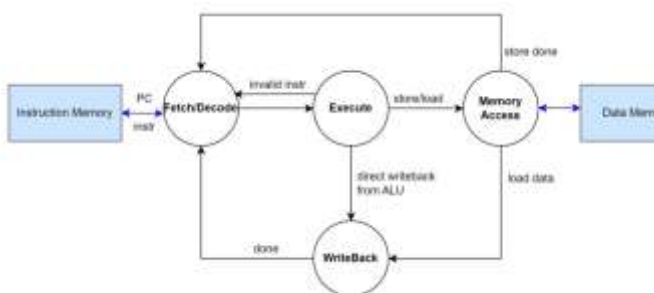
(IF = Instruction Fetch, ID = Instruction Decode, EX = Execute, MEM = Memory access, WB = Register write back).

SYSTEM ARCHITECTURE

Main Stages:

- Stage 1: Instruction Fetch (IF)
- Stage 2: Instruction Decode (ID)
- Stage 3: Execute (EX)
- Stage 4: Memory Access (MEM)
- Stage 5: Write Back (WB)

Data Flow:



EXPERIMENTAL SETUP / SIMULATION

Tools Used:

- Icarus Verilog
- GTKWave

Verification:

The simulation waveform confirms that the processor correctly fetched, decoded, executed, and wrote back the instruction results. The forwarding unit successfully resolved data dependencies and the pipeline operated without unnecessary stalls. The final value stored in register x3 was 5, matching the expected result.

RESULTS

Input Program

ADDI x1,x0,2

ADDI x2,x0,3

ADD x3,x1,x2

Expected Output

x1 = 2

x2 = 3

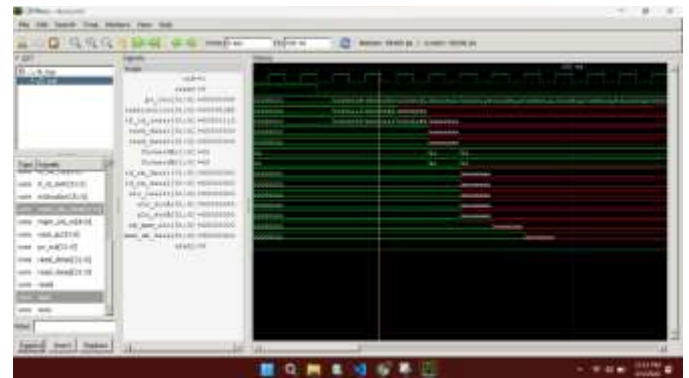
x3 = 5

Obtained Output

x1 = 2

x2 = 3

x3 = 5



3. CONCLUSION

The increasing demand for high-performance and energy-efficient computing systems has created the need for advanced processor architectures capable of executing instructions quickly and efficiently. Pipelining is one of the most effective techniques used in modern processors to improve instruction throughput and maximize hardware utilization. In this project, a 5-Stage Pipelined RISC-V Processor with Hazard Detection and Forwarding Unit has been successfully designed and implemented using Verilog HDL.

The proposed processor architecture consists of the five classical pipeline stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). To enhance processor performance and ensure

correct instruction execution, pipeline registers were incorporated between consecutive stages. These registers enable multiple instructions to be processed simultaneously, thereby increasing instruction-level parallelism and improving overall system throughput.

One of the major challenges in pipelined processors is the occurrence of data hazards. To address this issue, a Forwarding Unit was implemented to forward data directly between pipeline stages, reducing unnecessary pipeline stalls and minimizing execution delays. In addition, a Hazard Detection Unit was integrated to identify instruction dependencies and generate stall signals whenever forwarding alone could not resolve the hazard. These mechanisms significantly improved pipeline efficiency and ensured reliable processor operation.

The complete processor design was simulated and verified using Icarus Verilog and GTKWave. Simulation results demonstrated correct Program Counter operation, successful instruction fetching and decoding, proper ALU execution, accurate data propagation through pipeline registers, and successful operation of forwarding and hazard handling mechanisms. The processor correctly executed the test instruction sequence:

ADDI x1, x0, 2

ADDI x2, x0, 3

ADD x3, x1, x2

and produced the expected result:

x1 = 2

x2 = 3

x3 = 5

The obtained results confirm the functional correctness of the proposed processor architecture. Compared with conventional pipelined implementations, the inclusion of forwarding and hazard detection mechanisms reduces pipeline stalls, improves instruction throughput, and enhances processor performance without significantly increasing hardware complexity.

Therefore, the proposed 5-Stage Pipelined RISC-V Processor with Hazard Detection and Forwarding Unit provides an efficient and scalable processor architecture suitable for FPGA-based embedded systems, educational processor design, and advanced computer architecture research. The developed processor also serves as a strong foundation for future enhancements such as branch prediction, cache memory integration, superscalar execution, out-of-order processing, and FPGA implementation on Xilinx or Intel development boards.

Key Achievements

- Designed and implemented a complete 5-stage pipelined RISC-V processor.
- Successfully developed IF, ID, EX, MEM, and WB stages.
- Implemented pipeline registers for parallel instruction execution.
- Integrated Forwarding Unit to reduce data hazards.
- Integrated Hazard Detection Unit to improve pipeline reliability.

- Verified processor functionality through simulation.
- Achieved correct instruction execution and data propagation.
- Improved processor throughput compared to basic non-pipelined architectures.
- Developed a scalable architecture suitable for future research and FPGA implementation.

4.ACKNOWLEDGEMENT

I express my sincere gratitude to Andhra University, Visakhapatnam, for providing me the opportunity to undertake this M.Tech project entitled "Design and Implementation of a 5-Stage Pipelined RISC-V Processor with Hazard Detection and Forwarding Unit Using Verilog HDL."

I would like to convey my heartfelt thanks to my project guide for providing valuable guidance, encouragement, constructive suggestions, and continuous support throughout the development of this project. Their technical expertise and motivation played a vital role in the successful completion of this work.

I extend my sincere thanks to the Head of the Department, Department of Electronics and Communication Engineering, for providing the necessary facilities, infrastructure, and academic environment required for carrying out this research work.

I am also grateful to all the faculty members of the Department of Electronics and Communication Engineering for their valuable suggestions, technical discussions, and encouragement during the course of this project.

I would like to acknowledge the support provided by my friends and classmates who contributed through discussions, suggestions, and assistance during various stages of the project implementation and testing.

Special thanks are extended to the developers and contributors of open-source tools such as Verilog HDL, Icarus Verilog, GTKWave, and Visual Studio Code, which were extensively used for the design, simulation, verification, and analysis of the proposed processor architecture.

I express my deepest gratitude to my parents and family members for their unconditional love, constant encouragement, moral support, and motivation throughout my academic journey. Their belief in my abilities has been the foundation of my success.

Finally, I thank everyone who directly or indirectly contributed to the successful completion of this project and helped me achieve this milestone in my M.Tech program.

REFERENCES

[1] David A. Patterson and John L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware Software Interface, 2nd Edition, Morgan Kaufmann Publishers, 2021.

[2] David A. Patterson and John L. Hennessy, Computer Architecture: A Quantitative Approach, 6th Edition, Morgan Kaufmann Publishers, 2019.

[3] John F. Wakerly, Digital Design: Principles and Practices, 5th Edition, Pearson Education, 2018.

[4] M. Morris Mano and Michael D. Ciletti, Digital Design with RTL Design, Verilog HDL and VHDL, Pearson Education, 2017.

[5] Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd Edition, Pearson Education, 2003.

[6] Andrew Waterman and Krste Asanović, The RISC-V Instruction Set Manual, Volume I: User-Level ISA, RISC-V Foundation, Version 2.2, 2019.

[7] Andrew Waterman, Yunsup Lee, David Patterson, and Krste Asanović, The RISC-V Instruction Set Manual Volume II: Privileged Architecture, RISC-V Foundation, 2019.

[8] K. Asanović and D. Patterson, "Instruction Sets Should Be Free: The Case for RISC-V," Electrical Engineering and Computer Sciences Department, University of California, Berkeley, Technical Report No. UCB/EECS-2014-146, 2014.

[9] A. Waterman, Y. Lee, D. Patterson, and K. Asanović, "The RISC-V Instruction Set Manual," University of California, Berkeley, 2019.

[10] M. Gautschi, P. D. Schiavone, A. Traber, and L. Benini, "Near-Threshold RISC-V Core with DSP Extensions for Scalable IoT Endpoint Devices," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, Vol. 25, No. 10, pp. 2700–2713, 2017.

[11] S. Mathew, M. Anders, H. Kaul et al., "A 0.19 pJ/b, 0.09 mm², 1.4 GHz RISC-V Core with Dynamic Branch Prediction," IEEE Journal of Solid-State Circuits, Vol. 53, No. 1, pp. 34–45, 2018.

[12] P. Mishra and N. Dutt, "Processor Description Languages: Applications and Methodologies," IEEE Computer Society Press, 2008.